

UNIVERSITÀ DEGLI STUDI DI SALERNO

CORSO DI LAUREA IN INFORMATICA
ESAME DI PROGRAMMAZIONE E STRUTTURE DATI

**SISTEMA GESTIONALE PER LA
MANUTENZIONE CONDOMINIALE**

DOCUMENTAZIONE DEL PROGETTO

*URCIUOLO ANDREA
VEROSIMILE ALESSANDRO
SORBO SAMUELE
CLASSE: PEU-Z*

ANNO ACCADEMICO 2025 – 2026
DATA DI CONSEGNA: 29 MAGGIO 2026

Indice

1	Motivazioni per le Strutture Dati	3
1.1	Motivazione della scelta: Albero Binario di Ricerca e Array per i Technician	3
1.2	Motivazione della scelta: Linked List per Intervention	4
1.3	Motivazione della scelta: Priority Queue e Array per le Request	5
2	Guida all'Utilizzo del Software	6
2.1	Avvio del Programma	6
2.2	Parametri di Avvio	6
2.3	Interazione con l'Interfaccia	6
3	Progettazione	7
3.1	Moduli di definizione ADT	7
3.2	Moduli di supporto e Strutture Dati	7
3.3	Moduli operativi	7
4	Specifiche Sintattiche e Semantiche	8
4.1	ADT Request	8
4.1.1	Specifica Sintattica	8
4.1.2	Specifica Semantica	8
4.2	ADT Technician	10
4.2.1	Specifica Sintattica	10
4.2.2	Specifica Semantica	10
4.3	ADT Intervention	12
4.3.1	Specifica Sintattica	12
4.3.2	Specifica Semantica	12
4.4	Modulo OP_REQUEST	14
4.4.1	Specifica Sintattica	14
4.4.2	Specifica Semantica	14
4.5	Modulo OP_TECHNICIAN	16
4.5.1	Specifica Sintattica	16
4.5.2	Specifica Semantica	16
4.6	Modulo OP_INTERVENTION	18
4.6.1	Specifica Sintattica	18
4.6.2	Specifica Semantica	18
4.7	Struttura Btree	19
4.7.1	Specifica Sintattica	19
4.7.2	Specifica Semantica	19
4.8	Struttura PQueue	21
4.8.1	Specifica Sintattica	21
4.8.2	Specifica Semantica	21
4.9	ADT List	23
4.9.1	Specifica Sintattica	23
4.9.2	Specifica Semantica	23

5	Razionale dei Casi di Test	25
5.1	Casi Limite Globali (Trasversali a tutti i moduli)	25
5.2	Casi Limite Specifici per Modulo	26
5.2.1	Modulo Btree	26
5.2.2	Modulo List	26
5.2.3	Modulo PQueue	26
5.2.4	Modulo Request	27
5.2.5	Modulo Technician	27
5.2.6	Modulo Intervention	27
5.2.7	Modulo OpRequest	28
5.2.8	Modulo OpIntervention	28
6	Note Finali e Compatibilità	29

1 Motivazioni per le Strutture Dati

1.1 Motivazione della scelta: Albero Binario di Ricerca e Array per i Technician

La gestione dei Technician consiste in un sistema ibrido che combina un **Albero Binario di Ricerca (BST)**, ordinato per tipologia, e un **Array di puntatori**, indicizzato tramite ID. Questa architettura mira a massimizzare l'efficienza sia nella categorizzazione che nell'accesso diretto.

- **Organizzazione per Categoria (Albero Ordinato):** L'inserimento dei tecnici in un albero basato sulla tipologia (es. elettricisti, idraulici) garantisce una complessità di ricerca, inserimento e cancellazione mediamente pari a $O(\log n)$, ottimizzando le operazioni di filtraggio per categoria.
- **Accesso Diretto tramite ID (Array):** Per rendere immediata l'individuazione di un tecnico specifico, viene usato un array di supporto che memorizza i riferimenti ai nodi dell'albero. Grazie all'indicizzazione tramite ID, l'accesso ad un technician è garantito con una complessità di $O(1)$, eliminando la necessità di scorrere l'intera struttura.
- **Efficienza nella Visualizzazione:** L'utilizzo di un albero ordinato semplifica drasticamente la generazione di report per tipologia. Una visita in-order della struttura permette di ottenere un elenco già ordinato con complessità $O(n)$, eliminando così ulteriori operazioni di ordinamento che ridurrebbero l'efficienza.
- **Compromessi tra Memoria e Velocità:** Sebbene l'aggiunta dell'array comporti un lieve incremento nel consumo di memoria (causando un leggero overhead), il beneficio in termini di velocità di risposta del sistema è prioritario. Questa scelta rende il software estremamente reattivo sia nelle ricerche per tipologia che in quelle per ID, garantendo scalabilità anche al crescere del numero di technician.

1.2 Motivazione della scelta: Linked List per Intervention

La scelta di utilizzare due **Linked List** distinte per gestire gli interventi (Completati e Non Completati) è giustificata da una serie di punti:

- **Separazione delle Responsabilità e prestazioni:** La suddivisione in due strutture indipendenti permette di ottimizzare le operazioni di ricerca e stampa. Per generare report sugli interventi "in lavorazione" (frequenza medio-alta), il sistema non deve scorrere l'intera struttura, ma solo la lista pertinente, mantenendo la complessità a $O(n)$ dove n è solo il sottoinsieme degli interventi attivi.
- **Efficienza nell'Inserimento:** Data l'alta frequenza di registrazione, l'utilizzo della Linked List garantisce una complessità costante $O(1)$ all'inserimento in testa.
- **Logica di migrazione:** Una volta completata un'operazione, essa viene rimossa dalla lista degli incompleti ($O(1)$ se in posizione nota o testa) e inserita in quella dei completati ($O(1)$).
- **Ottimizzazione della Memoria:** L'uso di liste separate permette di conoscere lo stato dell'intervento implicitamente, eliminando flag booleani o variabili di controllo extra, riducendo l'overhead di memoria.

1.3 Motivazione della scelta: Priority Queue e Array per le Request

L'utilizzo di una **Priority Queue** per la gestione delle Request è la scelta più efficace per garantire la gestione delle urgenze in maniera efficiente. A questa struttura è stato affiancato un **Array di puntatori** per la gestione degli identificativi, al fine di massimizzare la reattività del sistema nell'accesso diretto.

- **Ordinamento Automatico per Urgenza:** La Priority Queue permette di mantenere le Request non assegnate in un ordine tale per cui l'elemento con priorità massima sia sempre il primo a essere estratto, indipendentemente dall'ordine cronologico di inserimento.
- **Efficienza nell'Assegnazione:** L'operazione di estrazione della Request più urgente (operazione ad alta frequenza nel sistema) avviene con una complessità di $O(\log n)$, garantendo prestazioni ottimali anche con un numero elevato di pendenze.
- **Accesso Rapido tramite ID (Array):** L'utilizzo di un array di supporto per memorizzare i riferimenti alle Request permette di rendere immediato l'accesso a una specifica richiesta tramite il suo identificativo. Questo garantisce una complessità di $O(1)$ per l'indirizzamento diretto, eliminando la necessità di scansioni lineari della struttura.
- **Compromessi di Visualizzazione:** La stampa dell'intera coda in ordine di priorità ha una complessità di $O(n \log n)$. Tale costo è considerato un compromesso accettabile data la bassa frequenza di visualizzazione totale rispetto alle operazioni di inserimento ed estrazione.

2 Guida all'Utilizzo del Software

2.1 Avvio del Programma

Per interagire con l'applicazione, è necessario innanzitutto aprire il terminale di sistema e posizionarsi all'interno della directory principale del progetto:

```
cd ManutenzioneCondominio_PSD
```

Una volta raggiunta la cartella, il software può essere avviato eseguendo il file principale tramite il seguente comando:

```
.\manutenzione [parametro]
```

2.2 Parametri di Avvio

In fase di esecuzione bisogna passare uno dei tre parametri a seconda della modalità desiderata:

--exe : Modalità Standard

Esegue l'applicazione normalmente, senza alcuna configurazione aggiuntiva.

--dev : Modalità Sviluppo

Prima di avviare l'interfaccia, il sistema alloca e popola automaticamente la memoria con un set di dati di test (100 request, 100 technician e 20 intervention). Questa modalità è pensata per velocizzare i test manuali e le interazioni durante lo sviluppo e correzione del codice.

--test : Modalità Testing

Avvia esclusivamente l'esecuzione delle test suite e mostra a schermo le strutture dati allocate per il testing. È una modalità progettata per facilitare il debugging e la verifica del corretto funzionamento delle singole funzioni.

2.3 Interazione con l'Interfaccia

Una volta avviato il programma in modalità standard (-exe) o sviluppo (-dev), l'interazione con l'utente avviene tramite un'interfaccia a riga di comando (CLI) intuitiva.

Il flusso logico è guidato da menu testuali: per eseguire un'operazione specifica o navigare tra le funzionalità del software, è sufficiente digitare il carattere corrispondente all'opzione desiderata visibile a schermo e premere *Invio*.

3 Progettazione

3.1 Moduli di definizione ADT

request.h:

Definisce l'ADT per le richieste. Gestisce i dati presi in input dall'utente per permettere la creazione di una richiesta valida.

technician.h:

Definisce l'ADT per i tecnici. Gestisce i dati presi in input dall'utente per permettere la creazione di un tecnico valido.

intervention.h:

Definisce l'ADT per gli interventi. Gestisce la creazione di un intervento in base alla richiesta più urgente ed il primo tecnico disponibile.

3.2 Moduli di supporto e Strutture Dati

item.h:

Questo modulo permette alle strutture dati di operare su puntatori void*, evitando la duplicazione di codice e permettendo che due liste diverse possano avere tipi di valori diversi.

list.h:

Implementa una lista semplicemente collegata. Nel software vengono create due liste per gli interventi completati e gli interventi in corso.

Pqueue.h:

Implementa una coda prioritaria. Viene usata per garantire la priorità di urgenza delle richieste.

Btree.h:

Implementa un albero binario di ricerca. Viene utilizzato per la gestione dei Technician, permettendo di organizzarli in base alla loro tipologia.

3.3 Moduli operativi

opRequest.h:

Fornisce le funzioni per manipolare la coda prioritaria creata nel main, permettendo la visualizzazione e filtraggio di dati.

opTechnician.h:

Fornisce le funzioni per manipolare l'albero dei tecnici creato nel main, permettendo la visualizzazione e filtraggio dei dati.

opIntervention.h:

Fornisce le funzioni per manipolare la lista dei tecnici creata nel main, e una funzione relativa alla ricerca di eventuali conflitti temporali nella creazione di nuovi interventi

4 Specifiche Sintattiche e Semantiche

4.1 ADT Request

4.1.1 Specifica Sintattica

Tipi coinvolti: request, int, char, string.

Operazione	Firma Sintattica
newRequest	(int) → request
getIdRequest	(request) → int
getType	(request) → char
getUrgency	(request) → int
getApartment	(request) → int
getSubmissionDate	(request) → string
getDescription	(request) → string
printRequest	(request) → void
deallocateRequest	(request) → void

4.1.2 Specifica Semantica

newRequest(id)

Pre-condizioni: Nessuna.

Post-condizione: Restituisce un puntatore ad una request inizializzata dall'utente via input. Se l'allocazione fallisce, restituisce NULL.

Complessità: $O(n)$, dove n è la lunghezza della stringa della descrizione.

getIdRequest(r)

Pre-condizioni: $r \neq \text{NULL}$.

Post-condizione: Restituisce un intero rappresentante l'identificativo della richiesta.

Complessità: $O(1)$.

getType(r)

Pre-condizioni: Nessuna.

Post-condizione: Se $r \neq \text{NULL}$ restituisce il carattere della tipologia, altrimenti 'z'.

Complessità: $O(1)$.

getUrgency(r)

Pre-condizioni: Nessuna.

Post-condizione: Se $r \neq \text{NULL}$ restituisce $n \in \{1, \dots, 5\}$, altrimenti -1 .

Complessità: $O(1)$.

getApartment(r)

Pre-condizioni: Nessuna.

Post-condizione: Se $r \neq \text{NULL}$ restituisce il numero appartamento (≥ 0), altrimenti -1 .

Complessità: $O(1)$.

getSubmissionDate(r)

Pre-condizioni: Nessuna.

Post-condizione: Se $r \neq \text{NULL}$ restituisce la data nel formato "YYYY/MM/DD", altrimenti NULL.

Complessità: $O(1)$.

getDescription(r)

Pre-condizioni: $r \neq \text{NULL}$.

Post-condizione: Restituisce il puntatore alla stringa contenente la descrizione di r .

Complessità: $O(1)$.

printRequest(r)

Pre-condizioni: $r \neq \text{NULL}$.

Post-condizione: Nessuna (void).

Side-effect: Stampa in output il riepilogo formattato di tutti i dati di r .

Complessità: $O(n)$, dove n è la lunghezza della stringa della descrizione.

deallocateRequest(r)

Pre-condizioni: $r \neq \text{NULL}$.

Post-condizione: Nessuna (void).

Side-effect: Dealloca la memoria precedentemente allocata per r .

Complessità: $O(1)$.

4.2 ADT Technician

4.2.1 Specifica Sintattica

Operazione	Firma Sintattica
createTechnician	(int) $\rightarrow$ technician
deleteTechnician	(technician) $\rightarrow$ void
printTechnician	(technician) $\rightarrow$ void
addIntervention	(technician) $\rightarrow$ void
getIdCode	(technician) $\rightarrow$ int
getInterventionCount	(technician) $\rightarrow$ int
getName	(technician) $\rightarrow$ string
getSpecialization	(technician) $\rightarrow$ char

4.2.2 Specifica Semantica

createTechnician(id)

Pre-condizioni: Nessuna.

Post-condizione: Restituisce un puntatore a technician inizializzato tramite input. NULL in caso di fallimento.

Complessità: $O(n)$, con n lunghezza massima del buffer di input.

deleteTechnician(t)

Pre-condizioni: $t \neq \text{NULL}$.

Post-condizione: Nessuna (void).

Side-effect: Dealloca la memoria puntata da t .

Complessità: $O(1)$.

printTechnician(t)

Pre-condizioni: $t \neq \text{NULL}$.

Post-condizione: Nessuna (void).

Side-effect: Stampa in output i dati del tecnico.

Complessità: $O(n)$, con n lunghezza della stringa nome.

addIntervention(t)

Pre-condizioni: $t \neq \text{NULL}$.

Post-condizione: Nessuna (void).

Side-effect: Incrementa di 1 il contatore interventi nel record di t .

Complessità: $O(1)$.

getIdCode(t)

Pre-condizioni: Nessuna.

Post-condizione: Se $t \neq \text{NULL}$ restituisce l'ID, altrimenti -1.

Complessità: $O(1)$.

getInterventionCount(t)

Pre-condizioni: Nessuna.

Post-condizione: Se $t \neq \text{NULL}$ restituisce $n \geq 0$, altrimenti -1.

Complessità: $O(1)$.

getName(t)

Pre-condizioni: Nessuna.

Post-condizione: Se $t \neq \text{NULL}$ restituisce il puntatore al nome, altrimenti NULL.

Complessità: $O(1)$.

getSpecialization(t)

Pre-condizioni: Nessuna.

Post-condizione: Se $t \neq \text{NULL}$ restituisce la specializzazione, altrimenti 'z'.

Complessità: $O(1)$.

4.3 ADT Intervention

4.3.1 Specifica Sintattica

Operazione	Firma Sintattica
<code>newIntervention</code>	$(\text{request}, \text{technician}) \rightarrow \text{intervention}$
<code>getRequestIntervention</code>	$(\text{intervention}) \rightarrow \text{request}$
<code>getTechnicianIntervention</code>	$(\text{intervention}) \rightarrow \text{technician}$
<code>getDateAppointment</code>	$(\text{intervention}) \rightarrow \text{string}$
<code>getTimeAppointment</code>	$(\text{intervention}) \rightarrow \text{string}$
<code>printIntervention</code>	$(\text{intervention}) \rightarrow \text{void}$
<code>deallocateIntervention</code>	$(\text{intervention}) \rightarrow \text{void}$

4.3.2 Specifica Semantica

`newIntervention(r, t)`

Pre-condizioni: $r \neq \text{NULL}, t \neq \text{NULL}$.

Post-condizione: Restituisce una struttura inizializzata con `r`, `t` e dati temporali via input. `NULL` se fallisce.

Complessità: $O(n)$, con n lunghezza delle stringhe in input (data/ora).

`getRequestIntervention(inter)`

Pre-condizioni: $\text{inter} \neq \text{NULL}$.

Post-condizione: Restituisce la struttura `request` associata all'intervento.

Complessità: $O(1)$.

`getTechnicianIntervention(inter)`

Pre-condizioni: $\text{inter} \neq \text{NULL}$.

Post-condizione: Restituisce la struttura `technician` associata all'intervento.

Complessità: $O(1)$.

`getDateAppointment(inter)`

Pre-condizioni: $\text{inter} \neq \text{NULL}$.

Post-condizione: Restituisce il puntatore alla stringa della data.

Complessità: $O(1)$.

`getTimeAppointment(inter)`

Pre-condizioni: $\text{inter} \neq \text{NULL}$.

Post-condizione: Restituisce il puntatore alla stringa dell'orario.

Complessità: $O(1)$.

printIntervention(inter)

Pre-condizioni: $inter \neq \text{NULL}$.

Post-condizione: Nessuna (void).

Side-effect: Stampa completa dell'intervento (inclusi request e technician).

Complessità: $O(n)$, con n complessità di stampa di request e technician.

deallocateIntervention(inter)

Pre-condizioni: $inter \neq \text{NULL}$.

Post-condizione: Nessuna (void).

Side-effect: Dealloca la memoria puntata da `inter`.

Complessità: $O(1)$.

4.4 Modulo OP_REQUEST

4.4.1 Specifica Sintattica

Tipi coinvolti: PQueue, request, int, char, void.

Operazione	Firma Sintattica
printAllRequests	(PQueue) $\rightarrow$ void
printRequestById	(PQueue, int) $\rightarrow$ void
printRequestsByType	(PQueue, char) $\rightarrow$ void
printRequestsByApartment	(PQueue, int) $\rightarrow$ void
getRequestById	(PQueue, int) $\rightarrow$ request

4.4.2 Specifica Semantica

printAllRequests(pq)

Pre-condizioni: Nessuna.

Post-condizione: Nessuna (void).

Side-effect: Estrae ciclicamente il massimo dalla coda, lo stampa e lo inserisce in una coda temporanea, per poi ripristinare la pq originale.

Complessità: $O(n \log n)$, dove n è il numero di elementi di pq.

printRequestById(pq, id)

Pre-condizioni: $pq \neq \text{NULL}$.

Post-condizione: Nessuna (void).

Side-effect: Scorre l'intera coda tramite estrazioni e reinserimenti per trovare la richiesta con l'ID specificato e ne stampa i dettagli.

Complessità: $O(n \log n)$, dove n è il numero di elementi di pq.

printRequestsByType(pq, type)

Pre-condizioni: $\text{type} \in \{ 'a', 'b', 'c', 'd', 'e', 'f' \}$.

Post-condizione: Nessuna (void).

Side-effect: Stampa tutte le richieste il cui tipo corrisponde al carattere type. Stampa un messaggio di errore se il tipo non è valido.

Complessità: $O(n \log n)$, dove n è il numero di elementi di pq.

printRequestsByApartment(pq, apartment)

Pre-condizioni: $\text{apartment} \geq 0$.

Post-condizione: Nessuna (void).

Side-effect: Filtra e stampa le richieste per l'appartamento indicato, ripristinando la coda al termine.

Complessità: $O(n \log n)$, dove n è il numero di elementi di pq.

getRequestById(pq, id)

Pre-condizioni: $pq \neq \text{NULL}$.

Post-condizione: Restituisce il puntatore alla request con l'ID corrispondente, se presente nella coda; altrimenti restituisce NULL.

Side-effect: Estrae temporaneamente gli elementi dalla coda di priorità originale per scansionarli e li inserisce in una coda ausiliaria. Al termine della ricerca, ripristina integralmente la struttura originale pq e dealloca la struttura temporanea.

Complessità: $O(n \log n)$, dove n è il numero di elementi presenti in pq.

4.5 Modulo OP_TECHNICIAN

4.5.1 Specifica Sintattica

Tipi coinvolti: Btree, technician, char, int, void.

Operazione	Firma Sintattica
printAllTechnicians	(Btree) $\rightarrow$ void
printTechniciansBySpecialization	(Btree, char) $\rightarrow$ void
printTechniciansByWorkload	(Btree) $\rightarrow$ void
printMostActiveTechnician	(Btree) $\rightarrow$ void
findTechnicianByType	(Btree, char) $\rightarrow$ technician

4.5.2 Specifica Semantica

printAllTechnicians(T)

Pre-condizioni: Nessuna.

Post-condizione: Nessuna (void).

Side-effect: Effettua una visita in-order dell'albero e stampa i dettagli di ogni tecnico. La stampa risulterà ordinata per tipologia.

Complessità: $O(n)$, dove n è il numero di nodi in T .

printTechniciansBySpecialization(T, spec)

Pre-condizioni: $spec \in \{'a', 'b', 'c', 'd', 'e', 'f'\}$.

Post-condizione: Nessuna (void).

Side-effect: Individua e stampa tutti i tecnici con la specializzazione indicata.

Complessità: $O(n)$, dove n è il numero di nodi in T

printTechniciansByWorkload(T)

Pre-condizioni: $T \neq \text{NULL}$.

Post-condizione: Nessuna (void).

Side-effect: Alloca dinamicamente un array temporaneo, vi inserisce i tecnici tramite una visita dell'albero e li ordina in modo decrescente in base al numero di interventi completati (utilizzando il Bubble Sort). Infine, stampa la classifica e dealloca l'array.

Complessità: $O(n^2)$, dove n è il numero di nodi in T .

printMostActiveTechnician(T)

Pre-condizioni: T non vuoto.

Post-condizione: Nessuna (void).

Side-effect: Scansiona l'intero albero per identificare il tecnico che ha completato il maggior numero di interventi e ne stampa i dettagli.

Complessità: $O(n)$, dove n è il numero di nodi in T .

findTechnicianByType(T, spec)

Pre-condizioni: Nessuna.

Post-condizione: Restituisce il puntatore al primo tecnico trovato con la specializzazione corrispondente; restituisce NULL se non esiste un tecnico della tipologia corrispondente o se l'albero è vuoto.

Side-effect: Nessuno.

Complessità: $O(n)$, dove n è il numero di nodi di T.

4.6 Modulo OP_INTERVENTION

4.6.1 Specifica Sintattica

Tipi coinvolti: list, intervention, int, char, string, boolean.

Operazione	Firma Sintattica
printAllInterventions	(list) $\rightarrow$ void
printInterventionById	(list, int) $\rightarrow$ void
printInterventionsByDate	(list, string) $\rightarrow$ void
printInterventionsByTech	(list, int) $\rightarrow$ void
printInterventionsByType	(list, char) $\rightarrow$ void
checkConflict	(list, int, string, string) $\rightarrow$ boolean

4.6.2 Specifica Semantica

printAllInterventions(l)

Pre-condizioni: Nessuna.

Post-condizione: Nessuna (void).

Side-effect: Scorre l'intera lista e stampa ogni intervento. Se vuota, stampa un messaggio di avviso.

Complessità: $O(n)$, dove n è il numero di interventi nella lista.

printInterventionById(l, requestId)

Pre-condizioni: requestId > 0 .

Post-condizione: Nessuna (void).

Side-effect: Stampa il primo intervento trovato con ID richiesta corrispondente. Se non trovato, errore.

Complessità: $O(n)$, dove n è il numero di elementi nella lista.

printInterventionsByDate(l, date)

Pre-condizioni: date nel formato "YYYY/MM/DD".

Post-condizione: Nessuna (void).

Side-effect: Stampa tutti gli interventi la cui data coincide con l'input.

Complessità: $O(n)$, dove n è il numero di elementi nella lista.

checkConflict(l, techId, date, time)

Pre-condizioni: techId > 0 , date e time formattati correttamente.

Post-condizione: Restituisce 1 (true) se esiste già un intervento per quel tecnico in quel momento, 0 (false) altrimenti.

Complessità: $O(n)$, dove n è il numero di elementi nella lista.

4.7 Struttura Btree

4.7.1 Specifica Sintattica

Tipi coinvolti: Btree, node, technician, int, void.

Operazione	Firma Sintattica
newBtree	(void) $\rightarrow$ Btree
emptyBtree	(Btree) $\rightarrow$ int
figlioSX	(Btree) $\rightarrow$ Btree
figlioDX	(Btree) $\rightarrow$ Btree
consBtree	(technician, Btree, Btree) $\rightarrow$ Btree
getRoot	(Btree) $\rightarrow$ node*
getItem	(node*) $\rightarrow$ technician
setItem	(node*, technician) $\rightarrow$ void
insertTechnician	(Btree, technician) $\rightarrow$ Btree
sizeBtree	(Btree) $\rightarrow$ int
clearTreeNodees	(Btree) $\rightarrow$ void

4.7.2 Specifica Semantica

newBtree()

Pre-condizioni: Nessuna.

Post-condizione: Restituisce un albero binario vuoto.

Complessità: $O(1)$.

emptyBtree(T)

Pre-condizioni: Nessuna.

Post-condizione: Restituisce 1 se l'albero T è vuoto, 0 altrimenti.

Complessità: $O(1)$.

figlioSX(T)

Pre-condizioni: $T \neq \text{NULL}$.

Post-condizione: Restituisce il puntatore al sottoalbero sinistro di T .

Complessità: $O(1)$.

figlioDX(T)

Pre-condizioni: $T \neq \text{NULL}$.

Post-condizione: Restituisce il puntatore al sottoalbero destro di T .

Complessità: $O(1)$.

consBtree(val, sx, dx)

Pre-condizioni: Nessuna.

Post-condizione: Restituisce un nuovo nodo contenente *val* con figli *sx* e *dx*. Restituisce NULL in caso di fallimento dell'allocazione.

Complessità: $O(1)$.

getRoot(T)

Pre-condizioni: $T \neq \text{NULL}$.

Post-condizione: Restituisce il puntatore alla radice dell'albero.

Complessità: $O(1)$.

getItem(N)

Pre-condizioni: $N \neq \text{NULL}$.

Post-condizione: Restituisce la struttura *tecnici* memorizzata nel nodo *N*.

Complessità: $O(1)$.

setItem(N, tech)

Pre-condizioni: $N \neq \text{NULL}$ e $\text{tech} \neq \text{NULL}$.

Post-condizione: Aggiorna il valore del nodo *N* con il tecnico fornito.

Complessità: $O(1)$.

insertTechnician(T, newTech)

Pre-condizioni: Nessuna.

Post-condizione: Restituisce un nuovo albero contenente *newTech*. L'inserimento segue la logica BST basata sulla specializzazione.

Side-effect: Alloca un nuovo nodo e ricostruisce il percorso di inserimento.

Complessità: $O(h)$, dove *h* è l'altezza dell'albero.

sizeBtree(T)

Pre-condizioni: Nessuna.

Post-condizione: Restituisce il numero totale di nodi nell'albero.

Complessità: $O(n)$, dove *n* è il numero di nodi.

clearTreeNodes(T)

Pre-condizioni: Nessuna.

Post-condizione: Dealloca tutti i nodi dell'albero dalla memoria.

Side-effect: Libera la memoria di ogni nodo (non dealloca il tecnico associato).

Complessità: $O(n)$, dove *n* è il numero di nodi.

4.8 Struttura PQueue

4.8.1 Specifica Sintattica

Tipi coinvolti: PQueue, request, int, void.

Operazione	Firma Sintattica
newPQ	$(\text{void}) \rightarrow \text{PQueue}$
emptyPQ	$(\text{PQueue}) \rightarrow \text{int}$
getMax	$(\text{PQueue}) \rightarrow \text{request}$
deleteMax	$(\text{PQueue}) \rightarrow \text{int}$
insert	$(\text{PQueue}, \text{request}) \rightarrow \text{int}$
deallocatePQ	$(\text{PQueue}) \rightarrow \text{void}$

4.8.2 Specifica Semantica

newPQ()

Pre-condizioni: Nessuna.

Post-condizione: Alloca e restituisce una nuova coda di priorità vuota. Restituisce NULL in caso di fallimento dell'allocazione.

Side-effect: Alloca memoria per la struttura dati.

Complessità: $O(1)$.

emptyPQ(q)

Pre-condizioni: Nessuna.

Post-condizione: Restituisce 1 se la coda è vuota o non inizializzata, 0 altrimenti.

Complessità: $O(1)$.

getMax(q)

Pre-condizioni: $q \neq \text{NULL}$ e la coda non deve essere vuota.

Post-condizione: Restituisce l'elemento con il massimo valore di urgenza senza rimuoverlo. Restituisce NULL se la coda è vuota.

Complessità: $O(1)$.

deleteMax(q)

Pre-condizioni: $q \neq \text{NULL}$ e la coda non deve essere vuota.

Post-condizione: Rimuove l'elemento con la massima urgenza e ripristina la proprietà di Max-Heap. Restituisce 1 se l'operazione ha successo, 0 altrimenti.

Side-effect: Decrementa il conteggio degli elementi e modifica l'ordinamento interno.

Complessità: $O(\log n)$, dove n è il numero di elementi nella coda di priorità.

insert(*q*, *r*)

Pre-condizioni: $q \neq \text{NULL}$.

Post-condizione: Inserisce una nuova **request** nella coda e ripristina la proprietà di Max-Heap. Restituisce 1 in caso di successo, 0 in caso di errore.

Side-effect: Incrementa il numero di elementi e riorganizza la struttura interna (heapify-up).

Complessità: $O(\log n)$, dove n è il numero di elementi nella coda di priorità.

deallocatePQ(*q*)

Pre-condizioni: Nessuna.

Post-condizione: Rilascia tutta la memoria allocata per la struttura PQueue.

Side-effect: Libera la memoria della coda. Le singole strutture **request** non vengono deallocate.

Complessità: $O(1)$.

4.9 ADT List

4.9.1 Specifica Sintattica

Tipi coinvolti: list, item, int, void.

Operazione	Firma Sintattica
newList	$(\text{void}) \rightarrow \text{list}$
emptyList	$(\text{list}) \rightarrow \text{int}$
tailList	$(\text{list}) \rightarrow \text{list}$
consList	$(\text{list}, \text{item}) \rightarrow \text{list}$
getFirst	$(\text{list}) \rightarrow \text{item}$
sizeList	$(\text{list}) \rightarrow \text{int}$
reverseList	$(\text{list}) \rightarrow \text{list}$
insertList	$(\text{list}, \text{int}, \text{item}) \rightarrow \text{list}$
removeList	$(\text{list}, \text{int}) \rightarrow \text{list}$

4.9.2 Specifica Semantica

newList()

Pre-condizioni: Nessuna.

Post-condizione: Restituisce una nuova lista vuota.

Complessità: $O(1)$.

emptyList(l)

Pre-condizioni: Nessuna.

Post-condizione: Restituisce 1 se la lista l è vuota, 0 altrimenti.

Complessità: $O(1)$.

tailList(l)

Pre-condizioni: $l \neq \text{NULL}$ (la lista non deve essere vuota).

Post-condizione: Se $l = \langle a_1, a_2, \dots, a_n \rangle$, restituisce la coda della lista $l' = \langle a_2, \dots, a_n \rangle$.

Complessità: $O(1)$.

consList(l, val)

Pre-condizioni: Nessuna.

Post-condizione: Se $l = \langle a_1, a_2, \dots, a_n \rangle$, restituisce la lista aggiornata con l'elemento inserito in testa $l' = \langle \text{val}, a_1, a_2, \dots, a_n \rangle$. Se l'allocazione fallisce, gestisce l'errore o restituisce la lista originale.

Complessità: $O(1)$.

getFirst(l)

Pre-condizioni: $l \neq \text{NULL}$ (la lista non deve essere vuota).

Post-condizione: Se $l = \langle a_1, a_2, \dots, a_n \rangle$, restituisce il primo elemento a_1 .

Complessità: $O(1)$.

sizeList(l)

Pre-condizioni: Nessuna.

Post-condizione: Restituisce un intero $n \geq 0$ che rappresenta il numero totale di elementi presenti nella lista.

Complessità: $O(n)$, dove n è il numero di elementi.

reverseList(l)

Pre-condizioni: Nessuna.

Post-condizione: Se $l = \langle a_1, a_2, \dots, a_n \rangle$, restituisce una nuova lista invertita $l' = \langle a_n, \dots, a_2, a_1 \rangle$.

Complessità: $O(n)$, dove n è il numero di elementi.

insertList(l, p, val)

Pre-condizioni: $\text{sizeList}(l) \geq p$ e $p \geq 0$.

Post-condizione: Se $l = \langle a_1, a_2, a_3, \dots, a_n \rangle$ e $p = 2$, restituisce la lista modificata con l'inserimento dell'elemento nella posizione indicata $l' = \langle a_1, a_2, \text{val}, a_3, \dots, a_n \rangle$.

Complessità: $O(p)$, nel caso peggiore $O(n)$ se l'elemento si trova in coda.

removeList(l, p)

Pre-condizioni: $\text{sizeList}(l) > p$ e $p \geq 0$.

Post-condizione: Rimuove il nodo alla posizione p . Se $p = k$ e $l = \langle a_1, \dots, a_k, \dots, a_n \rangle$, restituisce la lista priva dell'elemento rimosso $l' = \langle a_1, \dots, a_{k-1}, a_{k+1}, \dots, a_n \rangle$.

Side-effect: Dealloca la memoria occupata dal nodo rimosso.

Complessità: $O(p)$, nel caso peggiore $O(n)$ se l'elemento si trova in coda.

5 Razionale dei Casi di Test

5.1 Casi Limite Globali (Trasversali a tutti i moduli)

Per evitare ridondanze, i seguenti scenari vengono applicati a tutte le strutture dati e alle relative funzioni contenute nel software:

Puntatori Nulli ed Entità non Inizializzate (NULL)

Test: Chiamata di qualsiasi funzione, getter, visualizzatore o distruttore passando un puntatore pari a NULL (T, l, q, r, tech, inter).

Razionale: Verificare la stabilità dei controlli di guardia iniziali. Il sistema deve intercettare il puntatore nullo e terminare l'esecuzione in sicurezza (ritornando valori sentinella come 0, -1, NULL, 'z' o stampando messaggi di cortesia), azzerando il rischio di Segmentation Fault.

Strutture Dati Vuote

Test: Chiamata di funzioni di calcolo, ricerca o filtraggio su strutture appena allocate ma prive di elementi (emptyBtree, emptyList, emptyPQ).

Razionale: Garantire che i cicli o le discese ricorsive si interrompano immediatamente alla condizione base senza tentare accessi a memoria non allocata.

Fallimento di Allocazione Memoria (malloc / realloc pari a NULL)

Test: Simulazione del fallimento della memoria di sistema durante le operazioni di inserimento o creazione.

Razionale: Verificare che il software risponda restituendo NULL o 0 (segnalazione di fallimento) in modo controllato, preservando intatti i dati preesistenti ed evitando dereferenziazioni pericolose o memory leak strutturali.

Ricerche senza Corrispondenze

Test: Chiamata di funzioni di filtraggio o ricerca passando parametri geometricamente corretti, ma non associati a nessuna entità in memoria (ID, date o specializzazioni inesistenti).

Razionale: Assicurarsi che l'intero spazio dei dati venga scansionato, che le strutture vengano ripristinate correttamente e che il sistema notifichi l'assenza di risultati senza sollevare eccezioni.

5.2 Casi Limite Specifici per Modulo

5.2.1 Modulo Btree

Inserimento Valori Duplicati

Test: Inserimento di un tecnico con specializzazione identica a un nodo già presente nell'albero.

Razionale: Verificare il rispetto della condizione $\leq$, accertandosi che il duplicato venga instradato correttamente nel sottoalbero sinistro senza corrompere l'albero.

Albero Degenere (Sbilanciato a Lista)

Test: Inserimento di tecnici con specializzazioni in sequenza strettamente crescente o decrescente.

Razionale: Verificare che la ricorsione profonda dello scenario peggiore ($O(n)$) non causi fenomeni di Stack Overflow nelle funzioni ricorsive (`insertTechnician`, `clearTreeNodees`).

Buffer Overflow nel Visualizzatore

Test: Chiamata di `showTree` su un albero con altezza elevata (> 10).

Razionale: Verificare che la stringa locale `newPrefix[256]` non superi la sua capacità durante la concatenazione ricorsiva dei prefissi grafici a terminale.

5.2.2 Modulo List

Inserimento/Rimozione Fuori Limite (Index Out of Bounds)

Test: Chiamata di `insertList` o `removeList` con indice $p > \text{sizeList}(l)$ o $p < 0$.

Razionale: Verificare il rispetto delle precondizioni: l'algoritmo deve ignorare l'operazione restituendo la lista originale intatta senza causare crash o alterare i puntatori.

5.2.3 Modulo PQueue

Saturazione dell'Array e Ridimensionamento (`numel == capacity`)

Test: Inserimento del 51° elemento (avendo `INITIAL_CAPACITY_PQ = 50`).

Razionale: Verificare che il meccanismo di `realloc` raddoppi la capacità del vettore senza corrompere i dati e gestisca gli indici in modo coerente con la convenzione dell'indice 0 non utilizzato.

Elementi con Identica Priorità (Urgenza Uguale)

Test: Inserimento di più richieste aventi lo stesso valore di `getUrgency`.

Razionale: Verificare che le condizioni di arresto nelle procedure di `heapify up` ($>$) e `down` ($\geq$) gestiscano i duplicati in modo stabile, senza generare cicli infiniti o scambi superflui.

5.2.4 Modulo Request

Input Utente non Valido (Fuori Range)

Test: Inserimento di tipologie non censite, urgenze errate (es. 0, 6, stringhe) o numeri civici negativi.

Razionale: Verificare la tenuta dei cicli di validazione interattiva (`while` e `do-while`), accertandosi che il software svuoti il buffer di input, rifiuti il dato e lo richieda senza bloccarsi.

Caso Speciale Ascensore (`type == 'e'`)

Test: Creazione di una richiesta con tipologia 'e'.

Razionale: Verificare che la logica di business assegni automaticamente l'appartamento 0 (intero edificio) bypassando la richiesta di input del numero civico.

Validità Confini Stringa Data (`submissionDate`)

Test: Inserimento di stringhe con formato errato (es. con trattini) o date inesistenti (es. "2026/02/30").

Razionale: Assicurarsi che lo `scanf("%10s", ...)` impedisca il buffer overflow e che `checkDateValidity` intercetti le anomalie del calendario prima della copia con `strncpy`.

Descrizione al Limite della Capacità del Buffer

Test: Inserimento di una descrizione vuota o di lunghezza superiore a 512 caratteri.

Razionale: Verificare che l'uso di `fgets` con limite `sizeof(buffer)` gestisca le stringhe lunghe troncandole in modo sicuro, respingendo al contempo le stringhe vuote.

5.2.5 Modulo Technician

Nome Tecnico al Limite della Capacità del Buffer

Test: Inserimento di un nome vuoto o superiore a 50 caratteri.

Razionale: Verificare che la funzione `readString` (o `strncpy`) gestisca il limite dell'array statico `name[50]`, inserendo il terminatore `\0` in posizione 49 ed evitando corruzioni dello stack.

Specializzazione non Valida

Test: Inserimento in `createTechnician` di un carattere non mappato (es. `'\n'`).

Razionale: Verificare che l'ADT memorizzi il carattere in modo sicuro senza mandare in crash il sistema, delegando l'integrità del dominio al modulo controllore.

5.2.6 Modulo Intervention

Violazione delle Precondizioni di Legame

Test: Chiamata di `newIntervention` o `buildIntervention` passando una richiesta o un tecnico pari a `NULL`.

Razionale: Verificare che il costruttore interrompa immediatamente l'allocazione restituendo `NULL`, impedendo la creazione di record associativi orfani.

5.2.7 Modulo OpRequest

Invarianza dell'Ordine Interno dopo il Doppio Travaso

Test: Ispezione dello stato del Max-Heap prima e dopo una funzione di ricerca.

Razionale: Poiché gli elementi vengono estratti da pq e inseriti in una coda temporanea tempPq, l'ordine originale viene invertito. Il test accerta che il ciclo di ripristino finale ricostruisca le proprietà originarie del Max-Heap al 100% senza scambi di priorità desiderati.

5.2.8 Modulo OpIntervention

Sovrapposizione Parziale dei Dati di Pianificazione (checkConflict Negativo)

Test: Chiamata di checkConflict con stesso tecnico e data ma orario diverso, oppure stessa data e orario ma tecnico diverso.

Razionale: Verificare la precisione della logica booleana (AND dei criteri di tempo). La funzione deve ritornare 0 (nessun conflitto), evitando falsi positivi che bloccherebbero erroneamente l'operatività del sistema.

6 Note Finali e Compatibilità

Nota sulla compatibilità del sistema

Il file `Makefile` e l'intero software sono stati sviluppati e testati esclusivamente su sistema operativo Windows. Di conseguenza, non viene garantito il corretto funzionamento della compilazione e dell'eseguibile su piattaforme diverse (come macOS o distribuzioni Linux).